CQRS — Command Query Responsibility Segregation

Category: Data / messaging **One-liner:** Split a system's operations into commands (writes) and queries (reads) with separate models, so each can be optimised, scaled, and evolved independently.

Part 0 — Concepts you need first

What is CRUD?

Create, Read, Update, Delete — the four basic data operations. Most applications start here: one `User` class, one `users` table, one set of methods. CRUD is simple and usually right; this chapter is about when it stops being enough.

What is a "model"?

The code and data structures representing part of your **domain** (the business area) — the `User` class, its **validation** rules, its table, the code that loads and saves it. CQRS's central claim: **one model serving both reads and writes ends up serving neither well**, because their requirements are opposed.

Reads vs. writes — why they differ

The intuition the pattern rests on:

	Writes	Reads
Must be	Correct, validated, secure	Fast
Volume	Usually lower	Usually far higher
Complexity	Business rules (domain rules), constraints (database-enforced rules)	Joining (combining tables) and formatting
Cost of error	Severe — corrupt data	Mild — a stale (out-of-date) display
Optimal storage	Normalised (no duplication)	Denormalised (pre-joined)

The last row is the deep conflict. **Normalised** storage keeps each fact once — great for writes (update one row). **Denormalised** storage duplicates data into ready-made shapes — great for reads (no **joins** at query time). **One schema can't be both**; every design compromises — and CQRS says stop compromising, have two.

What is a DTO?

A **Data Transfer Object** — a plain object that just carries data, with no **business logic**. CQRS's read side returns DTOs shaped for the screen rather than domain objects.

What does "side effect" mean?

A state change that persists after a method returns. Reading has none; saving does. CQRS formalises this distinction.

Part 1 — The problem

A **monolith** usually has **one database for everything**, simultaneously serving **complex join queries** and **CUD** (Create, Update, Delete) operations; as the application grows, **handling both becomes really hard to manage.**

The scale mismatch

Complex applications need many **concurrent** operations, diverse data needs and efficient scaling, and plain CRUD struggles. **E-commerce:** thousands of users browsing (**reads**) while inventory updates, orders and new products arrive (**writes**). **Reads are far more numerous; writes carry more complexity** — and with one model you can't scale them independently.

The optimisation conflict — the banking example

- **Transactions** (deposits, withdrawals) — writes that must be **accurate and secure**, since they move money.
- **Balance inquiries** — reads that must be **quick**, in **real time**.

> **The trap:** optimise transactions for speed and security and balance inquiries slow down; optimise for quick balance checks and you compromise transaction efficiency or security. **One model can't be tuned for both** — it becomes hard to maintain, scale, and perform under different kinds of **load**.

Part 2 — The solution

- **Command** — an instruction to *do* something: create, update, delete (a deposit or withdrawal).
- **Query** — a request to *retrieve* data **without changing it** (checking your balance).
- **Responsibility Segregation** — handling commands (writes) separately from queries (reads).

The two sides

The Command Side handles all writes (processing transactions). **The Query Side** handles all reads, optimised for fast retrieval (showing the balance).

The formal rule

Worth memorising — it gives the pattern its rigour. As a **separation of concerns**, **every method is either a query or a command, never both:**

	Returns a value?	Changes state?
Query	✅ Returns a result	❌ No state change, no side effects
Command *(also called modifiers or mutators — methods that change data)*	❌ Only an indication of success or failure	✅ Changes system state

> **Why "never both" matters.** A method that changes state *and* returns data can't be called safely: you can't retry it (it might apply twice), can't **cache** it, can't reason about it. The split makes queries freely repeatable and cacheable, and commands explicitly consequential.

Why this is useful

- **Optimization** — tune each side: command side for security and accuracy, query side for speed.
- **Simplicity** — each side developed and maintained separately.
- **Scalability** — scale reads and writes independently by demand.
- **Flexibility in technology choice** — different **stacks** per side: a **relational database** (tables linked by keys, SQL) with constraints and **transactions** for writes; a **document store** (self-contained JSON-like documents) or **search index** (e.g. Elasticsearch) for reads.

In essence: reading and writing have different needs, and CQRS lets you treat them separately.

Part 3 — Architecture: three levels

CQRS is a spectrum, and the costs differ enormously by level — always say which you mean.

Level 1 — Separate models, one database

The Command Model handles writes and guards the **integrity and consistency** of data: receives commands, validates, applies business rules, changes state. **The Query Model** handles reads, returning current state efficiently. (The course's image: a disciplined librarian shelving every book correctly vs. a friendly one fetching what you need.)

The simplest useful form — two models, one database. **Most teams should stop here:** most of the design clarity, almost none of the operational cost.

Level 2 — Separate read and write databases

Separate databases add flexibility and independent scaling, but

> ⚠️ **significantly increase complexity**: you must propagate changes from the write database to the read database.

The problems it creates:

- **No atomic transactions across the read and write models** — so you rely on **asynchronous messaging and events** for **eventual consistency** (the read side catches up shortly — § Consistency models)
- **Out-of-order event propagation**
- **Event loss**
- **Sync discrepancies**
- **Read Model update failures**
- **The timing of UI updates after writing** — what does the user see right after hitting save, if the read model hasn't caught up?

It demands **a deep understanding of distributed transactions** and suits you **only if your non-functional requirements** (speed, scale, reliability — not features) **genuinely demand it.**

Level 3 — Event Sourcing

Exists to solve one Level 2 problem: **ordering**. With separate databases kept in sync eventually, **the sequence of events from write side to read side is crucial**; rapid successive writes **received out of order leave the read model outdated** — and **many asynchronous message buses prioritise availability and performance and don't guarantee delivery order.**

Event Sourcing stores the write model as an **append-only event store** (events only ever added, never edited or deleted) recording every action; a model's state is rebuilt by **replaying its event history**. Each write-model instance is an **independent event stream**, which enables:

- **Various data views**, by replaying events differently
- **Rebuilding read models** from scratch from the write side's events — the direct fix for sync problems
- **Simplified auditing** — the streams record every change to every model
- **New read models** built by replaying past events — new views of history that never existed before

> **However:** event sourcing **adds complexity and is a significant undertaking**, especially if you're new to CQRS, event sourcing, or distributed systems. Don't adopt Level 3 because you adopted Level 1 — separate decisions with very different costs.

Part 4 — Inner workings

The journey of a command

A command expresses an **intention to change state** — e.g. `UpdateUserAddress`:

1. **Validation** — **data format, permissions, business rules**: is the address valid, may this user change it? Fail → **rejected**.
2. **Handling** — the **command handler**, the command side's workhorse, updates state.
3. **Generating events** — the change is represented as an **event**, `UserAddressUpdated`, stored in the **event store** as a historical record.

The query's quest

`GetUserAddress` is just a data request. **Queries need no validation and generate no events** — the **query handler** fetches and returns data.

> That asymmetry follows directly from the formal rule: a query changes nothing, so there's nothing to validate or record — which is why the read path can be much simpler and faster.

Synchronising the two sides

Event Sourcing keeps them in step: the `UserAddressUpdated` event is **replayed on the query side**, updating its state, so the separately-run command and query sides **stay in sync**.

Part 5 — Worked example: user profile management

Create a user (command), **update their email** (command), **view user details** (query).

The pieces

Side	Piece	Role
Command (write) model	**Command class**	An action to perform, e.g. `CreateUserCommand` (`userId`, `name`, `email`)
	Command handler	Takes the command, runs business logic, persists changes
	Domain model	The state of the data — the `User` class
	Repository	Data **persistence** (saving and loading)
Query (read) model	**DTO**	The data to read, e.g. `UserProfileDTO`
	Query class	A query, e.g. `GetUserProfileQuery` (`userId`)
	Query handler	Returns the data, often from **read-optimised storage**
Infrastructure	**Command bus / Query bus**	Route commands / queries to their handlers
	Event store	In advanced setups, stores the events resulting from commands

Command classes

```
public class CreateUserCommand {
    private String userId;
    private String name;
    private String email;
}

public class UpdateUserEmailCommand {
    private String userId;
    private String newEmail;
}
```

Just data — intent expressed as an object, with no behaviour. Making intent an explicit object is part of the point: you can log it, queue it, or replay it.

Command handlers

```
public class CreateUserCommandHandler {
    private UserRepository userRepository;

    public CreateUserCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(CreateUserCommand command) {
        User user = new User(command.getUserId(), command.getName(), command.getEmail());
        userRepository.save(user);
    }
}

public class UpdateUserEmailCommandHandler {
    private UserRepository userRepository;

    public UpdateUserEmailCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(UpdateUserEmailCommand command) {
        if (!isValidEmail(command.getNewEmail())) {
            throw new IllegalArgumentException("Invalid email format");
        }
        User user = userRepository.findById(command.getUserId());
        if (user != null) {
            user.setEmail(command.getNewEmail());
            userRepository.save(user);
        }
    }

    private boolean isValidEmail(String email) {
        return email != null && email.contains("@");
    }
}
```

Notice: `handle` returns `void` — commands don't return data (the formal rule). And **validation lives here**, before any state change — the write side's "accuracy and security" focus made concrete.

Domain model and repository

```
public class User {
    private String id;
    private String name;
    private String email;
}

public interface UserRepository {
    void save(User user);
    User findById(String id);
}
```

The read side

```
public class UserProfileDTO {
    private String id;
    private String name;
    private String email;
}

public class GetUserProfileQuery { private String userId; }
public class ReadUserEmailQuery  { private String userId; }
```

```
public class GetUserProfileQueryHandler {
    private UserRepository userRepository;

    public UserProfileDTO handle(GetUserProfileQuery query) {
        User user = userRepository.findById(query.getUserId());
        return new UserProfileDTO(user.getId(), user.getName(), user.getEmail());
    }
}

public class ReadUserEmailQueryHandler {
    private UserRepository userRepository;

    public String handle(ReadUserEmailQuery query) {
        User user = userRepository.findById(query.getUserId());
        return user != null ? user.getEmail() : null;
    }
}
```

Granularity: `ReadUserEmailQuery` returns just a `String`. Read models can be **as narrow as the screen needs** — each query gets a purpose-built shape instead of one bloated shared object. And handlers return **DTOs, not domain objects**: the read side doesn't expose the write model.

Infrastructure

```
public class SimpleCommandBus implements CommandBus {
    private CreateUserCommandHandler handler;

    @Override
    public void dispatch(CreateUserCommand command) {
        handler.handle(command);
    }
}

public class SimpleQueryBus implements QueryBus {
    private GetUserProfileQueryHandler handler;

    @Override
    public UserProfileDTO dispatch(GetUserProfileQuery query) {
        return handler.handle(query);
    }
}

public class SimpleEventStore implements EventStore {
    private List<UserEvent> events = new ArrayList<>();

    @Override
    public void storeEvent(UserEvent event) {
        events.add(event);
    }
}
```

The buses route to handlers; the event store captures changes that, in a full application, can be **replayed to rebuild an entity's state** (an entity = one business object, e.g. one user).

> **Where this sits: Level 1** — both sides share `UserRepository` and one database; the event store is unused by reads. Not a flaw: Level 1 is where most real systems should live and already delivers CQRS's main benefit, design clarity.

Part 6 — Issues, considerations, and performance

Complexity

Maintaining two models is genuinely complex, a roadblock for developers new to the pattern, with **a steep learning curve**.

Managing data consistency

The scenario: the command model changes before the query model is updated; **a query arriving in that interval returns outdated data.**

> The central trade-off of CQRS — raise it in interviews. In UI terms: a user saves their profile, the page reloads, and the old name is still there. Mitigations: the UI displays **optimistically** (shows the expected result before the server confirms), or reads right after a write go to the write model (Part 12C).

Handling transactions

The scenario: several commands form one logical transaction and one fails — **how do you roll back the earlier commands' changes?** Commands change state, and there's no simple erase. *(Exactly what Saga addresses with compensating transactions.)*

Dealing with asynchronicity

CQRS often brings async operations, making **error handling and debugging harder — finding an error's source is challenging**.

The performance upside

Segregating commands and queries lets you optimise each side; if queries far outnumber commands, **give the query side more resources** (more trains on the busiest routes).

Part 7 — Use cases

- **High-performance applications** — the most compelling case: **reads far outweigh writes**, e.g. millions browsing products vs. far fewer purchases; optimise for the read volume.
- **Complex business logic** — separating responsibilities simplifies **state-changing logic** (division of labour on an assembly line): the write model can be rich with rules and **invariants** (conditions that must always hold, e.g. "balance never negative") without slowing or complicating reads.
- **Collaborative domains** — **many users updating the same data** (a shared document). **Treating updates as commands that can fail or be rejected by rules** makes conflicts manageable.

Part 8 — System design examples

Event-driven microservices

Orders move through **placed, paid, shipped, delivered** — each a **command-driven state change** — while other microservices **query** order state (shipment tracking, customer notifications). CQRS gives a cleaner, more efficient design.

Real-time analytics

A social platform where users constantly **post updates (commands)** while the platform serves **real-time engagement analytics (queries)**. Their needs are opposite: **commands must be fast and reliable**; **queries need heavy computation but aren't time-critical**. Segregation **allocates resources to each need** — a textbook fit, since the two **workloads** would harm each other sharing one model.

Part 9 — Common beginner misconceptions

- **"CQRS requires two databases."** That's Level 2. Level 1 — two models, one database — is where most systems should live.
- **"CQRS requires event sourcing."** Frequently paired, but independent choices; CQRS can have no events at all.
- **"CQRS is the same as read replicas." Read replicas** are synced copies of the *same* database to spread read **load**; CQRS is a *different model* shaped for reading. You can use both.
- **"Commands are just HTTP POSTs."** A command is a design concept, not a **transport** detail — an explicit object representing intent, which is what lets you queue, log and replay it.
- **"It's always faster."** It adds **indirection**. Faster when read and write workloads genuinely conflict; pure overhead when they don't.
- **"Apply it to the whole application."** Almost always wrong. Apply it where read and write needs genuinely diverge; leave simple CRUD as CRUD.

Part 10 — Check your understanding

1. Why can't one database schema be optimal for both reads and writes?

2. What are the two halves of the formal rule, and why does "never both" matter?
3. Which level of CQRS should most teams adopt, and why?
4. What specific problem does event sourcing solve that Level 2 has?
5. What is the central trade-off, expressed as something a user would notice?
6. Why is the social media analytics example a textbook fit?

(Answers: 1 — writes want normalised storage where each fact lives once, reads want denormalised pre-joined shapes, and these are directly opposed; 2 — queries return data and change nothing, commands change state and return only success or failure; "never both" makes queries safely repeatable and cacheable and makes commands explicitly consequential; 3 — Level 1, two models over one database, because it gives the design clarity with almost none of the operational cost; 4 — out-of-order event delivery leaving the read model stale, solved by replaying an append-only event stream; 5 — the consistency window: a user saves their profile and the page still shows the old value; 6 — posting must be fast and reliable while analytics is computation-heavy but not time-critical, so the two workloads would degrade each other if they shared a model.)

Part 11 — Interview soundbite (30 seconds)

> "CQRS separates writes from reads — commands that change state and return nothing, queries that return data and change nothing. Reads and writes usually have opposite needs: in banking, transactions need accuracy and security while balance inquiries need speed, and at the storage level writes want normalised data while reads want denormalised, pre-joined shapes — one model can't be tuned for both. It's a spectrum. Level 1 is two models over one database — where most systems should stop: design clarity, almost no operational cost. Level 2 is separate read and write databases: independent scaling, but no atomic transactions across them, so eventual consistency via async events with out-of-order delivery and lost messages to handle. Level 3 is event sourcing: the write side is an append-only event log and read models are rebuilt by replay — fixing ordering and giving audit history and brand-new read models from past events, but a big undertaking and a separate decision. The main downside throughout is the consistency window — save your profile and still see the old name — plus complexity and harder debugging. It pairs with event-driven architecture, and with sagas when one logical transaction spans several commands."

Part 12 — Implementation

A. Runnable demo

Full source: `demos/CqrsDemo.java`

```
cd demos && ./gradlew runCh12
```

The formal rule made mechanical — command handlers return `void`:

```
void handle(ChangeEmail cmd) {                    // <- returns void
    User u = writeStore.get(cmd.userId());
    if (!cmd.newEmail().contains("@")) throw new IllegalArgumentException("invalid email");
    String old = u.email;
    u.email = cmd.newEmail();
    publish(new UserEvent.EmailChanged(cmd.userId(), old, cmd.newEmail()));
}

UserProfileView get(String id) { return store.get(id); }   // <- returns data, changes nothing
```

Rebuilding a read model from the event log — the Level 3 superpower:

```
fast.clear();                              // wipe the read model entirely
eventStore.all().forEach(fast::apply);     // replay history
```

B. What you should see

```
=== 3. Level 2: separate read store -> CONSISTENCY WINDOW ===
  write store immediately: carol@new.com
  read model immediately:  carol@old.com   <- STALE
  read model after 300ms:  carol@new.com   <- converged

=== 4. Several read models from ONE write ===
  profile view : UserProfileView[id=u4, name=Dave, email=dave@x.com]
  directory view: [UserDirectoryEntry[name=Dave, initial=D], ...]

=== 5. Level 3: rebuild the read model by replaying events ===
  after wiping the read model:  null
  after replaying the log:      UserProfileView[id=u1, name=Alice, ...]
```

Section 3 is the trade-off to describe in an interview, made observable: the write store has the new email while the read model still has the old — not a bug, the deal you accepted. Section 5 is the compensating power: **read models are disposable**, because the event log is the truth.

C. Production — Axon Framework

(**Axon**: a Java framework for CQRS and event sourcing. An **aggregate** is a cluster of objects changed as one unit — here, one user; annotations such as `@CommandHandler` map messages to methods.)

```
@Aggregate
public class UserAggregate {

    @AggregateIdentifier private String userId;
    private String email;

    @CommandHandler
    public UserAggregate(CreateUserCommand cmd) {
        if (!cmd.email().contains("@")) throw new IllegalArgumentException("invalid email");
        AggregateLifecycle.apply(new UserCreatedEvent(cmd.userId(), cmd.name(), cmd.email()));
    }

    @CommandHandler
    public void handle(ChangeEmailCommand cmd) {
        if (cmd.newEmail().equals(this.email)) return;              // no-op, no event
        AggregateLifecycle.apply(new EmailChangedEvent(cmd.userId(), this.email, cmd.newEmail()));
    }

    // State is derived by applying events - never assigned directly
    @EventSourcingHandler
    public void on(UserCreatedEvent e) { this.userId = e.userId(); this.email = e.email(); }

    @EventSourcingHandler
    public void on(EmailChangedEvent e) { this.email = e.newEmail(); }
}
```

The projection — the read side (a **projection** builds a read model by applying events):

```java
@Component
public class UserProfileProjection {

    private final UserProfileRepository repo;

    @EventHandler
    public void on(UserCreatedEvent e) {
        repo.save(new UserProfileView(e.userId(), e.name(), e.email()));
    }

    @EventHandler
    public void on(EmailChangedEvent e) {
        repo.findById(e.userId()).ifPresent(v -> { v.setEmail(e.newEmail()); repo.save(v); });
    }

    @QueryHandler
    public UserProfileView handle(FindUserProfileQuery q) {
        return repo.findById(q.userId()).orElseThrow();
    }
}
```

Plain Spring, no framework — Level 1, where most systems should live (**JPA**, the Java Persistence API, maps classes to tables; a **native query** is hand-written SQL):

```java
// Write side: JPA against a normalised schema
public interface UserRepository extends JpaRepository<User, String> {}

// Read side: a projection interface or native query against a denormalised view
public interface UserProfileReadRepository extends Repository<UserProfileView, String> {
    @Query(value = """
        SELECT u.id, u.name, u.email, COUNT(o.id) AS order_count
        FROM users u LEFT JOIN orders o ON o.user_id = u.id
        WHERE u.id = :id GROUP BY u.id
        """, nativeQuery = true)
    UserProfileView findProfile(@Param("id") String id);
}
```

> **You don't need Axon, Kafka or event sourcing to do CQRS.** Two repositories and two model classes over one database is legitimate CQRS and delivers most of the benefit.

Handling the consistency window in the UI — three practical options:

```java
// 1. Read-your-own-writes: route reads right after a write to the write model
if (recentlyWrittenBy(userId)) return writeModel.find(userId);
return readModel.find(userId);

// 2. Return the new state from the command endpoint so the UI can render optimistically
// 3. Return a version token; the client polls the read model until it catches up
```

(A **version token** is a number identifying the write; to **poll** is to ask repeatedly until ready.)

D. Implementation pitfalls

1. Jumping straight to Level 3. Event sourcing is a large commitment — event schema evolution, **snapshotting** for long streams, replay tooling. Adopting it *because* you adopted CQRS is the most common way teams get into trouble. Start at Level 1.

2. Commands that return data. Once `handle()` returns the updated entity the rule collapses and you're back to one model. Return void, or at most an ID.

3. Business logic in the read model. Projections only *shape* data; any deciding rule belongs on the write side, or the sides will disagree.

4. Ignoring the consistency window in the UI. Users who save and see stale data file bugs. Design for it with one of the three techniques above.

5. Not being able to rebuild projections. Disposable read models are event sourcing's main benefit — only if replay is *tested*. Make rebuilding routine, not theoretical.

6. Unbounded event streams. An aggregate with 100,000 events takes forever to load by replay. Use **snapshots** — persist state every N events and replay only from there.

7. Applying CQRS everywhere. Simple CRUD entities stay CRUD. Apply it where read/write needs genuinely diverge, and be able to say *why* for each place.

8. Events named as commands. `UpdateEmail` is a command; `EmailChanged` is an event. Wrong tense signals events that are disguised instructions, reintroducing **coupling** (Event-Driven Part 0).

Related patterns

- **Event-Driven Architecture** — the substrate that syncs command and query sides
- **Saga** — for transactions spanning multiple commands, where rollback isn't automatic
- **BFF** — read models shaped per client is the same instinct as BFF applied to data